

Design Decisions

Transformation:

I focused the transformation stage on making the data consistent and safe for storage.

- **QueryRecord (Cleaning):** I used this to filter out records missing a `transaction_id`. To test the flow's stability, I set default values for missing data: **'all'** for categories, **'product1'** for products, and **1000** for prices. which is not correct in real world but I just did that to ensure the dataflow is going well.
- **UpdateRecord (Enrichment):** I added a `submitted_date` to every record. This provides a clear timestamp for when the data was processed, which is vital for auditing.
- **MergeRecord (Optimization):** I used this to bundle individual records into larger files. This is more efficient for HDFS than storing thousands of tiny files, which can slow down the cluster.
- **DeduplicateRecord (Integrity):** This is the final check to catch and remove duplicate `transaction_ids`, ensuring only unique records are stored in the data lake.

Applying NiFi Best Practices

I configured the following settings to ensure the pipeline stays stable and handles errors automatically.

- **Back Pressure:** Set to **10,000 objects**. This acts as a "safety valve" that pauses ingestion if the HDFS sink becomes too slow, preventing the system from running out of memory.
- **Yield Duration:** Configured to **1 second**. This tells a processor to wait briefly if it hits an error, preventing it from constantly retrying and wasting CPU resources.
- **Retry & Penalization:** I set a **30-second Penalization**. If a file fails to write, it is moved to the back of the line for 30 seconds, allowing temporary network issues to resolve.
- **Prioritizers:** I used **OldestFlowFileFirst**. This ensures the data is processed in the order it was received, maintaining the correct chronological sequence.